

Backend Learning — Reference Notes

Bookstore Full-Stack Platform | Compiled from chat | Items tracked: 3

1. Navigation Properties in EF Core

What it is

A navigation property is a property on an entity class that lets you traverse to a related entity (or collection of entities) as an object, instead of manually joining on a foreign key. EF Core uses it to build relationships and generate JOINS behind the scenes.

Analogy

Like a contact entry that stores a direct link to the spouse's contact card, not just an ID number — you can jump straight to the related record instead of looking it up separately.

Example

```
public class Author
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;

    // Collection navigation property -- one Author has many Books
    public List<Book> Books { get; set; } = new();
}

public class Book
{
    public int Id { get; set; }
    public string Title { get; set; } = string.Empty;

    // Foreign key -- scalar, maps to a real column
    public int AuthorId { get; set; }

    // Reference navigation property -- one Book has one Author
    public Author Author { get; set; } = null!;
}
```

How / why it works

- `Book.Author` and `Author.Books` aren't columns -- EF Core infers the FK relationship from `AuthorId` (convention-based) and uses nav properties for `.Include()` eager loading.
- Reference navigation property (`Author Author`) points to a single related entity. Collection navigation property (`List<Book> Books`) points to many.
- Without `.Include()`, accessing a nav property on a disconnected entity throws or returns null (unless lazy loading is explicitly enabled -- off by default, generally avoided due to N+1 risk).

Production relevance

This is the exact reconciliation point for Stage C: EF Core's migration generates the same FK column (AuthorId) from these nav properties that was hand-written in Stage B's schema.

```
var books = await context.Books
    .Include(b => b.Author)
    .Where(b => b.Author.Name == "Tolkien")
    .ToListAsync();
```

2. FK Delete Constraint Behavior (SQL Server)

Scenario

What happens at the DB level when deleting a Customer who has existing Orders, with a foreign key constraint in place.

Analogy

Like trying to delete a customer's master file from a filing cabinet while their open invoice folders still reference that customer's ID — the cabinet won't let the master file be removed while other folders still point to it.

What happens

By default, SQL Server enforces referential integrity. If Orders.CustomerId has a FOREIGN KEY constraint pointing to Customers.Id with no special delete behavior specified, this fails:

```
DELETE FROM Customers WHERE Id = 5;
```

Error: 'The DELETE statement conflicted with the REFERENCE constraint "FK_Orders_Customers_CustomerId". The conflict occurred in database "BookstoreDb", table "dbo.Orders", column 'CustomerId'.' — the transaction rolls back entirely.

Why

The FK constraint's default ON DELETE behavior is NO ACTION — SQL Server checks child tables before allowing the parent row's deletion and refuses if any child rows reference it. This prevents orphaned records (e.g. an OrderItem pointing to a CustomerId that no longer exists).

Options and trade-offs

- **ON DELETE CASCADE** -- deleting the Customer auto-deletes their Orders (and further cascades). Dangerous here -- silently loses order history.
- **ON DELETE SET NULL** -- requires CustomerId to be nullable; orders become orphaned but preserved. Rarely right for financial records.
- **Soft delete** -- add an IsDeleted bit column to Customers instead of physically deleting. Standard real-world pattern for anything with order/audit history.
- **Delete Orders first, explicitly** -- only if that's the genuine intended business rule (rare for a bookstore).

Bookstore-specific note

Soft delete on Customers is the real answer for this project once Stage G's auditing/triggers territory comes up.

3. What Exactly Is an API

Analogy

An API is a restaurant menu + waiter, not the kitchen. A client doesn't walk into the kitchen and use the stove directly. It reads the menu (the API's defined contract — what requests are possible, what data to send, what comes back), tells the waiter what it wants, and the kitchen (the actual system, with all its internal complexity) does the work and returns a result. The internals of the kitchen stay invisible.

Definition

API = Application Programming Interface — a defined contract that lets one piece of software ask another piece of software to do something or provide data, without either side needing to know the other's internals.

In the Bookstore stack

- Angular (frontend) and BookstoreApi (.NET backend) are two separate programs -- separate processes, possibly separate machines. Angular doesn't know BookstoreApi is written in C#.
- The API is the contract between them: HTTP endpoints such as GET /api/books, POST /api/orders, DELETE /api/customers/5.
- Angular's HttpClient sends an HTTP request to an endpoint; BookstoreApi's controller (or Minimal API) handles it, possibly querying SQL Server, and returns a JSON response.
- Angular never touches SQL Server directly and doesn't know the Books table exists -- it only knows the shape of the JSON contract exposed by the API.
- SQL is not part of the API -- it is internal to the backend, hidden entirely from Angular via EF Core.

General case

APIs aren't limited to frontend/backend over a network. A class's public methods form an API for other code (e.g. Repository<T>'s public Add/GetById/Delete methods vs. its hidden internal implementation). Libraries, classes, OS system calls, and web services are all APIs at different scales and transports, sharing the same core idea: a stable, defined surface for interacting with something without needing to know how it works inside.

Why this matters for the project

This separation is exactly why Stage E/F work: Angular and BookstoreApi can be built, tested, and deployed independently, as long as both sides honor the same API contract. That independence is the point of the "full loop closed" milestone.